

SPRING BOOT · JAVA

Tareas Programadas y Carga de Archivos en Spring Boot

Implementación práctica para automatización y gestión de medios

Desarrollador Senior · Java & Spring Boot · Tema 1 & 2

Tareas Programadas — ¿Qué son y qué problemas resuelven?

Las **Tareas Programadas** (o **Cron Jobs**) son rutinas que se ejecutan automáticamente en momentos específicos o de forma periódica. El sistema se encarga de «despertar» y ejecutar la tarea **sin intervención humana**. Sirve para automatizar operaciones de mantenimiento o procesamiento por lotes en segundo plano.



Limpieza de datos

Borrar registros temporales o expirados de la base de datos de forma automática.



Notificaciones

Enviar correos de resumen o alertas a usuarios en intervalos regulares.



Reportes

Generar facturas mensuales o informes de actividad sin acción manual.



Sincronización

Consultar APIs externas periódicamente para mantener datos actualizados.

Tareas Programadas — Implementación en Spring Boot

Anotaciones clave

`@EnableScheduling`

Habilita el soporte de tareas programadas. Se coloca en la clase principal de la aplicación.

`@Scheduled`

Define cuándo se ejecuta un método. Admite `fixedRate`, `fixedDelay` o expresión cron.

Ejemplo: TareasAutomatizadas.java

`@Component`

`public class TareasAutomatizadas {`

`// Se ejecuta cada 10 segundos`

`@Scheduled(fixedRate = 10000)`

`public void reportarHoraActual() {`

`System.out.println(`

`"Hora actual: " + LocalDateTime.now()`

`);`

`}`

`// Cada día a las 08:00h`

`@Scheduled(cron = "0 0 8 * * *")`

`public void enviarResumenDiario() {`

`// lógica de negocio`

`}`

`}`

Carga de Archivos — Flujo y concepto

- .Es el proceso donde un cliente transfiere un archivo al servidor usando HTTP.
- .Sirve para recibir contenido del usuario y almacenarlo en disco, base de datos o la nube (S3).
- .Resuelve la necesidad de personalizar perfiles o adjuntar evidencia, convirtiendo una petición binaria en un objeto manejable en código.

Carga de Archivos — Implementación en Spring Boot

Ejemplo: ArchivoController.java

```
@RestController
@RequestMapping("/api/archivos")
public class ArchivoController {

    private final String DIR = "uploads/perfiles/";

    @PostMapping("/subir-foto")
    public ResponseEntity subirFoto(
        @RequestParam("file") MultipartFile archivo
    ) throws IOException {

        Path rutaDestino = Paths.get(
            DIR + archivo.getOriginalFilename()
        );

        Files.createDirectories(
            rutaDestino.getParent()
        );

        Files.copy(
            archivo.getInputStream(),
            rutaDestino,
            StandardCopyOption.REPLACE_EXISTING
        );

        return ResponseEntity.ok("Foto subida");
    }
}
```

Configuración en application.properties

Tamaño máximo por archivo

```
spring.servlet.multipart.max-file-size=5MB
```

Tamaño máximo total de petición

```
spring.servlet.multipart.max-request-size=10MB
```

Activar multipart

```
spring.servlet.multipart.enabled=true
```

(Al estar en `true`, Spring Boot activa una herramienta *interna* (llamada `MultipartResolver`) que toma esos ceros y unos y los transforma mágicamente en el objeto `MultipartFile` que usas en tu controlador) En las versiones modernas de Spring Boot, esta propiedad viene en `true` por defecto

¡NUEVO!

Mejores Prácticas y Puntos Clave

Tres consejos críticos que todo desarrollador debe conocer para evitar errores comunes en producción.



No olvidar el `@EnableScheduling`

Sin esta anotación en la clase principal, **las tareas simplemente no se ejecutarán**. Es un error silencioso difícil de detectar.

```
@SpringBootApplication
@EnableScheduling // ←
¡Obligatorio!
public class MiAplicacion { ... }
```



Seguridad de Archivos

Nunca confíes en el nombre original (`getOriginalFilename`). Usa UUIDs aleatorios para evitar sobrescrituras y ataques de **Directory Traversal**.

```
String nombreSeguro =
    UUID.randomUUID()
    + "_" +
    archivo.getOriginalFilename();
```



Manejo de Excepciones

Si se excede el tamaño máximo (`MaxUploadSizeExceededException`), usa `@ControllerAdvice` para capturar el error y **no romper la aplicación**.

```
@ControllerAdvice
public class
GlobalExceptionHandler {
    @ExceptionHandler(

        MaxUploadSizeExceededException.class)
    public ResponseEntity
    handleMaxSize(...) {
        return ResponseEntity
            .status(413).body("Archivo
            muy grande");
    }
}
```